You also have the option to send property values encrypted and let the applications themselves decrypt them, but that will require you to configure the symmetric key for all applications. You should also consider that decryption is not a cheap operation to perform.

Spring Cloud Config also supports encryption and decryption through asymmetric keys. This option provides more robust security than the symmetric alternative but it also increases complexity and maintenance costs due to key management tasks. In that case, you might want to consider relying on a dedicated secrets management solution. For example, you can use one of those offered by cloud providers and rely on the Spring Boot integration implemented by Spring Cloud, such as Azure Key Vault (Spring Cloud Azure), AWS Parameter Store or AWS Secrets Manager (Spring Cloud AWS), or Google Cloud Secret Manager (Spring Cloud GCP).

Should you prefer an open source solution, HashiCorp Vault (www.vaultproject.io) might be a good fit for you. It's a tool you can use to manage all your credentials, tokens, and certificates, both from a CLI and from a convenient GUI. You can integrate it directly with your Spring Boot applications using the Spring Vault project or add it as an additional backend for Spring Cloud Config Server.

For more information about secrets management in Spring, check out the official documentation for Spring Vault (https://spring.io/projects/spring-vault) and Spring Cloud Config (https://spring.io/projects/spring-cloud-config).

14.1.4 Disabling Spring Cloud Config

The next section will introduce a different way of configuring Spring Boot applications based on the native functionality provided by Kubernetes